Web Scraping (Beautiful Soup & rvest)

Lecture 13

Dr. Colin Rundel

Hypertext Markup Language

HTML is the language web pages are written in. It describes a page as nested elements marked by tags, which the browser renders into what we see. Web scraping is the reverse process, recovering the underlying data from that markup.

```
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <p align="center">Hello world!</p>
    <br/>
    <div class="name" id="first">John</div>
    <div class="name" id="last">Doe</div>
    <div class="contact">
      <div class="home">555-555-1234</div>
      <div class="home">555-555-2345</div>
      <div class="work">555-555-9999</div>
      <div class="fax">555-555-8888</div>
    </div>
  </body>
</html>
```

Anatomy of an element

An HTML document is built from elements. Most have an opening tag, optional attributes, content, and a closing tag. Void elements such as `br` and `img` have no content or closing tag.

```
1 <div class="name" id="first">John</div>
```

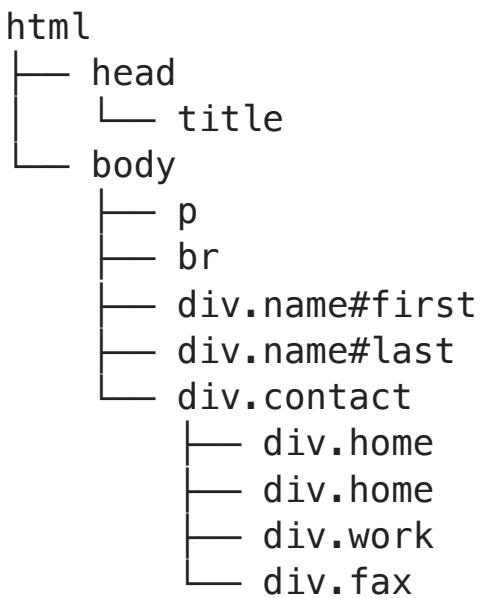
Part	Here	Notes
name	`div`	what kind of element this is (`p`, `a`, `table`, `div`, ...)
attribute(s)	`class="name"` `id="first"`	name and value pairs, any number of them
content	`John`	text, other elements, or both

Two attributes matter more than the rest when scraping:

- `class` - used to label a group of similar elements. Multiple elements can share a class, and one element can have multiple classes (`class="name bold"`).
- `id` - labels a single element and *should* be unique within a page.

The document tree

Elements nest inside one another, so a page is a tree. Selectors describe elements both by what they are and by where they sit in the tree.



- `html` is the root, an *ancestor* of everything else
- `body` is the *parent* of `p`, and `p` is a *child* of `body`
- `p`, `br` and the first three `div`s are *siblings*, since they share a *parent*
- `div.home` is a *descendant* of `body` but not a *child* of it

Beautiful Soup

Beautiful Soup is the most widely used Python package for parsing and navigating HTML. It is installed as `beautifulsoup4` and imported as `bs4`.

- `BeautifulSoup(html, parser)` - parse HTML from a string, returning the document as a `BeautifulSoup` object.

The document and every element in it are `Tag` objects, with methods and attributes for

- selecting - `.select()`, `.select_one()` by CSS selector, `.find_all()`, `.find()` by tag name and attributes.
- extracting - `.get_text()`, `.name`, `.attrs`, and `tag["attr"]` or `tag.get("attr")` for one attribute.
- traversing and modifying - moving between parents, children and siblings, and adding, replacing or removing elements. We will not cover these.

HTML & bs4

```py
html = '''<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <p align="center">Hello world!</p>
    <br/>
    <div class="name" id="first">John</div>
    <div class="name" id="last">Doe</div>
    <div class="contact">
      <div class="home">555-555-1234</div>
      <div class="home">555-555-2345</div>
      <div class="work">555-555-9999</div>
      <div class="fax">555-555-8888</div>
    </div>
  </body>
</html>'''
```

```py
from bs4 import BeautifulSoup

soup = BeautifulSoup(html, "html.parser")
soup.title
```

```
<title>This is a title</title>
```

```py
type(soup)
```

```
<class 'bs4.BeautifulSoup'>
```

```py
type(soup.title)
```

```
<class 'bs4.element.Tag'>
```

```py
type(soup.title.string)
```

```
<class 'bs4.element.NavigableString'>
```

Class	What it is
`BeautifulSoup`	the whole document, behaves like a `Tag`
`Tag`	an element, with a name, attributes and children
`NavigableString`	a run of text inside a tag, behaves like a `str`

Parsers

The second argument to `BeautifulSoup()` picks the parser. Real pages are often not well formed, and each parser repairs broken HTML differently. The result is the tree you will be selecting from, so pick one parser and use it consistently. `prettify()` shows the repaired tree as indented HTML.

```py
bad = "<p>One<p>Two<li>Three"
```

```py
BeautifulSoup(bad, "html.parser")
```

```
<p>One<p>Two<li>Three</li></p></p>
```

```py
print(
  BeautifulSoup(bad, "html.parser")
  .prettify()
)
```

```
<p>
 One
 <p>
  Two
  <li>
   Three
  </li>
 </p>
</p>
```

```py
BeautifulSoup(bad, "lxml")
```

```
<html><body><p>One</p><p>Two</p><li>Three</li></body></
```

```py
print(
  BeautifulSoup(bad, "lxml")
  .prettify()
)
```

```
<html>
 <body>
  <p>
   One
  </p>
  <p>
   Two
  </p>
  <li>
   Three
  </li>
 </body>
</html>
```

`"html.parser"` is built into Python. `"lxml"` is faster and `"html5lib"` is the most forgiving of broken HTML, but both need to be installed separately.

Tags as attributes

The simplest way to reach an element is by its tag name, as an attribute of the document or of another `Tag`. This returns the first element with that name, or `None` if there is none.

```py
soup.title
```

```
<title>This is a title</title>
```

```py
soup.p
```

```
<p align="center">Hello world!</p>
```

```py
soup.div
```

```
<div class="name" id="first">John</div>
```

```py
soup.body.div
```

```
<div class="name" id="first">John</div>
```

```py
print(soup.body.dv)
```

```
None
```

`soup.body.div` is shorthand for `soup.find("body").find("div")`. It is convenient for exploring a page, but it only ever

Selecting elements

`select()` returns all matches as a list, and `select_one()` returns the first matching `Tag`, or `None`. Both can be called on the whole document or on any `Tag`.

```py
soup.select("p")
```

```
[<p align="center">Hello world!</p>]
```

```py
p = soup.select_one("p"); p
```

```
<p align="center">Hello world!</p>
```

```py
soup.select_one(".contact").select("div")
```

```
[<div class="home">555-555-1234</div>,
 <div class="home">555-555-2345</div>,
 <div class="work">555-555-9999</div>,
 <div class="fax">555-555-8888</div>]
```

Many elements

The extraction methods belong to a single `Tag`. With more than one element you will need to use a comprehension.

```py
soup.select(".name").get_text()
```

```
AttributeError: ResultSet object has no attribute "get_text". You're probably treating a list of elements like a single element. Did you call find_all() when you meant to call find()?
```

```py
[d.get_text() for d in soup.select(".name")]
```

```
['John', 'Doe']
```

```py
[d["id"] for d in soup.select(".name")]
```

```
['first', 'last']
```

CSS selectors

CSS (Cascading Style Sheets) is the language used to style web pages. A stylesheet is a list of rules, and each rule starts with a selector that says which elements the styling applies to.

```css
div.name {
  color: blue;
  font-weight: bold;
}

#first {
  font-size: 120%;
}
```

Scraping tools borrow just the selector half of the language. It is a compact, standardized way of describing a set of elements, and it is already understood by a wide variety of tools.

Beautiful Soup's `select()` is implemented by the `soupsieve` package, which supports the CSS selector standard along

Basic selectors

The simplest selectors match on an element's tag name, its class, or its id. These can be written together with no spaces to require that all of them match, or separated by commas to match any of them.

Selector	Example	Description
element	`p`	Select all <p> elements
.class	`.name`	Select all elements with class="name"
#id	`#first`	Select the element with id="first"
element.class	`div.name`	Select all elements with class="name"
A, B	`p, #last`	Select everything matched by either selector

```py
soup.select(".name")
```

```
[<div class="name" id="first">John</div>, <div c
```

```py
soup.select("p, #last")
```

```
[<p align="center">Hello world!</p>, <div class=
```

```py
soup.select("#first")
```

```
[<div class="name" id="first">John</div>]
```

Combinators

Combinators join two selectors and match on the relationship between elements in the tree. The element that is returned is always the last one in the selector.

Selector	Example	Description
A B	`body div`	Select all elements anywhere inside <body> (descendant)
A > B	`body > div`	Select all elements directly inside <body> (child)
A + B	`.work + div`	Select the immediately after a `.work` element (adjacent sibling)
A ~ B	`.home ~ div`	Select all siblings that come after a `.home` element

```py
soup.select("body div")
```

```
[<div class="name" id="first">John</div>,
 <div class="name" id="last">Doe</div>,
 <div class="contact">
<div class="home">555-555-1234</div>
<div class="home">555-555-2345</div>
<div class="work">555-555-9999</div>
<div class="fax">555-555-8888</div>
</div>,
 <div class="home">555-555-1234</div>,
 <div class="home">555-555-2345</div>,
 <div class="work">555-555-9999</div>,
 <div class="fax">555-555-8888</div>]
```

```py
soup.select("body > div")
```

```
[<div class="name" id="first">John</div>,
 <div class="name" id="last">Doe</div>,
 <div class="contact">
<div class="home">555-555-1234</div>
<div class="home">555-555-2345</div>
<div class="work">555-555-9999</div>
<div class="fax">555-555-8888</div>
</div>]
```

```py
soup.select(".work + div")
```

```
[<div class="fax">555-555-8888</div>]
```

Attribute selectors

Any attribute can be used in a selector. This is most useful for links and images, e.g. `a[href^="https"]` or `img[src$=".png"]`.

Selector	Example	Description
`[attr]`	`[align]`	Select all elements with an align attribute
`[attr=value]`	`[class=fax]`	Select all elements where class is exactly “fax”
`[attr^=value]`	`[class^=ho]`	Select all elements where class starts with “ho”
`[attr$=value]`	`[id$=st]`	Select all elements where id ends with “st”
`[attr*=value]`	`[class*=or]`	Select all elements where class contains “or”

```py
soup.select("[align]")
```

```
[<p align="center">Hello world!</p>]
```

```py
soup.select("[class^=ho]")
```

```
[<div class="home">555-555-1234</div>, <div class="home
```

```py
soup.select("[id$=st]")
```

```
[<div class="name" id="first">John</div>, <div class="name" id="last">Doe</div>]
```

Pseudo-classes

Pseudo-classes start with a `:` and select on an element's position among its siblings or on a condition that the other selectors cannot express.

Selector	Example	Description
:first-child	`li:first-child`	Select elements that are the first child of their parent
:last-child	`li:last-child`	Select elements that are the last child of their parent
:nth-child(n)	`tr:nth-child(2)`	Select <tr> elements that are the second child of their parent
:nth-child(odd)	`tr:nth-child(odd)`	Select every other <tr>, also `even` and formulas like `3n+1`
:not(A)	`div:not(.ad)`	Select elements that do not match `.ad`
:has(A)	`div:has(img)`	Select elements that contain an <img>

Not every tool supports every pseudo-class, so a selector that works in the browser may fail in a scraping library. For example, the newer `details:open` works in Beautiful Soup but is an error in `rvest`.

See MDN's selector reference for the full list of selectors, combinators and pseudo-classes. Beautiful Soup also adds a

Pseudo-classes

```py
soup.select(".contact div:first-child")
```

```
[<div class="home">555-555-1234</div>]
```

```py
soup.select(".contact div:nth-child(2)")
```

```
[<div class="home">555-555-2345</div>]
```

```py
soup.select(".contact div:not(.home)")
```

```
[<div class="work">555-555-9999</div>, <div class="fax">555-555-8888</div>]
```

```py
soup.select("div:has(.work)")
```

```
[<div class="contact">
<div class="home">555-555-1234</div>
<div class="home">555-555-2345</div>
<div class="work">555-555-9999</div>
<div class="fax">555-555-8888</div>
</div>]
```

```py
soup.select(".contact div:-soup-contains('9999')")
```

```
[<div class="work">555-555-9999</div>]
```

Choosing a selector

There are usually many selectors that return the same elements. The goal is one that keeps working even if the page changes.

- Prefer classes and IDs that describe the content (`.price`, `#reviews`) over ones that describe position (`div > div:nth-child(3)`).
- Be as specific as needed and no more. A long chain of tags breaks the first time the site adds a `div`.
- Be wary of machine-generated class names like `.css-1q2w3e`, which tend to change with every update of the site.
- Anchor on a stable container, then select within it (`#results .title`).
- Always check how many elements came back. Too many or too few is the first sign that the selector is wrong.
- Not everything needs to be purely selector based - you can also process via Python or R after extraction.

Missing elements

When nothing matches, `select()` returns an empty list and `select_one()` returns `None`, so chaining a method onto a missing element is an error.

```py
soup.select("span")
```

```
[]
```

```py
print(soup.select_one("span"))
```

```
None
```

```py
soup.select_one("span").get_text()
```

```
AttributeError: 'NoneType' object has no attribute 'get_text'
```

Attributes behave similarly. `tag["attr"]` raises a `KeyError` if the attribute is missing, while `tag.get("attr")` returns `None`.

```py
print(p.get("class"))
```

```
None
```

`find()` and `find_all()`

Beautiful Soup also has its own search methods that match on tag name and attribute values. `find_all()` returns a list and `find()` returns the first match (or `None`). Both can be chained, as each `Tag` is searched in the same way as the full document.

```py
soup.find_all("div", class_="home")
```

```
[<div class="home">555-555-1234</div>, <div class="home">555-555-2345</div>]
```

```py
soup.find(id="last")
```

```
<div class="name" id="last">Doe</div>
```

```py
soup.find("div", class_="contact").find_all("div", class_="work")
```

```
[<div class="work">555-555-9999</div>]
```

```py
soup.find(id="first")["class"]
```

```
['name']
```

`class_` has a trailing underscore because `class` is reserved in Python. Since an element can have several classes,

More flexible searching

Where `find_all()` earns its place over `select()` is in what it accepts as a filter. The tag name and each attribute can be a string, a list of strings, a regular expression, or `True` to match any value, and `string=` searches the text instead of the tags.

```py
soup.find_all(["p", "title"])
```

```
[<title>This is a title</title>, <p align="center">Hello world!</p>]
```

```py
soup.find_all(class_=re.compile("^ho"))
```

```
[<div class="home">555-555-1234</div>, <div class="home">555-555-2345</div>]
```

```py
soup.find_all(string=re.compile("555"))
```

```
['555-555-1234', '555-555-2345', '555-555-9999', '555-555-8888']
```

A function can be used as well. It receives each `Tag` and should return `True` for a match.

```py
soup.find_all(lambda t: t.name == "div" and t.get_text().endswith("4"))
```

```
[<div class="home">555-555-1234</div>]
```

Extracting text

`get_text()` returns the raw text of a tag and all of its descendants, including the source whitespace. The `separator` and `strip` arguments control how the pieces are joined.

```py
contact = soup.select_one(".contact")
contact.get_text()
```

```
'\n555-555-1234\n555-555-2345\n555-555-9999\n555-555-8888\n'
```

```py
contact.get_text(" ", strip=True)
```

```
'555-555-1234 555-555-2345 555-555-9999 555-555-8888'
```

The `.stripped_strings` generator yields the same pieces one at a time, one per descendant string, with the whitespace removed.

```py
list(contact.stripped_strings)
```

```
['555-555-1234', '555-555-2345', '555-555-9999', '555-555-8888']
```

Extracting text

`.string` is the text of a tag that has exactly one string as its content. It works for simple elements such as `p` or `div.home`, but a tag that contains other elements has no single string, so `.string` is `None` and `get_text()` must be used instead.

```py
soup.p.string
```

```
'Hello world!'
```

```py
soup.select_one("div.home").string
```

```
'555-555-1234'
```

```py
print(contact.string)
```

```
None
```

Reading from the web

`BeautifulSoup()` does not accept a URL, so the page is downloaded first with `requests` and the text of the response is then parsed.

```py
import requests

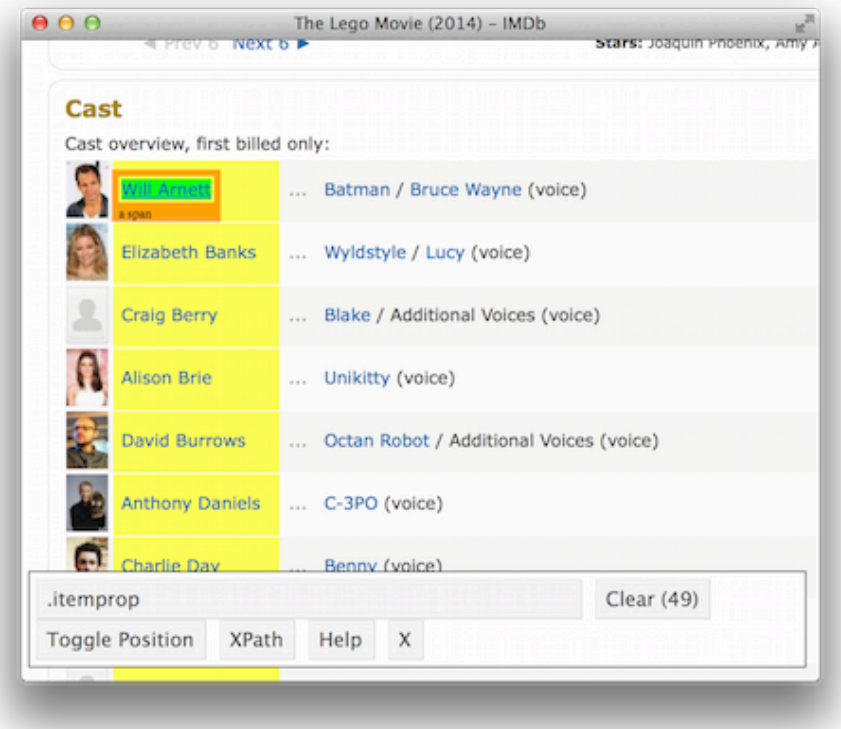
r = requests.get("https://example.com", timeout=10)
r.raise_for_status()

soup = BeautifulSoup(r.text, "html.parser")
soup.select(".title")
```

`raise_for_status()` turns a failed request (404, 500, …) into an exception. Without it we would happily parse the error page.

SelectorGadget

SelectorGadget is a browser extension (or bookmarklet) that builds a CSS selector interactively, by clicking on elements you want and elements you do not. When it cannot untangle a page, the browser's inspector (right-click, Inspect) shows the exact tags, classes and ids around any element.



selectorgadget.com

Web scraping considerations

“Can you?” vs “Should you?”

Researchers just released profile data on 70,000 OkCupid users without permission

By Brian Resnick | @B_resnick | brian@vox.com | May 12, 2016, 6:00pm EDT

A group of researchers has released a data set on nearly 70,000 users of the online dating site OkCupid. The data dump breaks the cardinal rule of social science research ethics: It took identifiable personal data without permission.

The information — while publicly available to OkCupid users — was collected by Danish researchers who never contacted OkCupid or its clientele about using it.

The data, collected from November 2014 to March 2015, includes user names, ages, gender, religion, and personality traits, as well as answers to the personal questions the site asks to help match potential mates. The users hail from a few dozen countries around the world.

The data dump did not reveal anyone's real name. But it's entirely possible to use clues from a user's location, demographics, and OkCupid user name to determine their identity.

> If your OkC username is one you've used anywhere else, I now know your sexual preferences & kinks, your answers to thousands of questions.
>
> — Scott B. Weingart (@scott_bot) May 11, 2016

“Can you?” vs “Should you?”



Crawl rules & `robots.txt`

Sites use the robots exclusion standard to publish crawler rules in `robots.txt`. Following these rules does not mean you are authorized to use the website to collect the data. Also consider the site's terms and the privacy of the people represented in the data.

You can find examples at all of your favorite websites: Google, Facebook, etc.

These files are meant to be machine readable, and `urllib.robotparser` from the standard library reads them. Check the rules for the same user agent that you will send with your requests.

```py
from urllib.robotparser import RobotFileParser

user_agent = "sta523-example"
rp = RobotFileParser("https://www.google.com/robots.txt")
rp.read()

rp.can_fetch(user_agent, "https://www.google.com/search?q=duke")
rp.can_fetch(user_agent, "https://www.google.com/maps")
```

These rules apply only to this site. Fetch a separate `robots.txt` when changing sites. An allowed path does not establish

Rate limiting

Making requests too quickly can overload a server, get your IP blocked, or violate a site's terms of service. Adding delays between requests is essential for responsible scraping.

The simplest approach is `time.sleep()` between requests. This loop uses `rp` from the previous slide, so all of the URLs must be on that site.

```py
import time

user_agent = "sta523-example"
urls = ["https://www.google.com/search?q=duke",
        "https://www.google.com/maps"]

for url in urls:
  if rp.can_fetch(user_agent, url):
    try:
      r = requests.get(url, headers={"User-Agent": user_agent}, timeout=10)
      r.raise_for_status()
    finally:
      time.sleep(5)
```

`finally` keeps the delay even if a request fails. `robots.txt` can also specify a `Crawl-delay`, available via

Graceful error handling

When scraping many pages, retrieval or parsing can fail. Wrapping the request in `try` / `except` lets us capture those errors and continue, returning `None` on failure.

```py
def safe_read(url):
  try:
    r = requests.get(url, timeout=10)
    r.raise_for_status()
    return BeautifulSoup(r.text, "html.parser")
  except requests.RequestException:
    return None

pages = [safe_read(url) for url in urls]
```

Keeping the exception alongside the result lets us inspect what went wrong afterwards.

```py
def safe_read(url):
  try:
    r = requests.get(url, timeout=10)
    r.raise_for_status()
    return {"result": BeautifulSoup(r.text, "html.parser"), "error": None}
  except requests.RequestException as e:
    return {"result": None, "error": e}
```

Example - Rotten Tomatoes

Open Movies at Home, sorted by popularity. Inspect the page and choose selectors that let us retrieve the following details:

- title,
- Tomatometer score,
- displayed critic-status label,
- absolute movie URL

Using these selectors construct a data frame containing these results. Some movies will be missing one or more of these, so use `None` for any missing fields.


rvest
www.rstudio.com

rvest

`rvest` is a package in the tidyverse that makes processing and manipulation of HTML data straightforward. It provides high-level functions for interacting with HTML via the `xml2` package.

- `read_html()` - read HTML data from a URL or character string.
- `html_elements()` / ~~`html_nodes()`~~ - select specified elements from the HTML document using CSS selectors (or XPath).
- `html_element()` / ~~`html_node()`~~ - select the first match within each input element using CSS selectors (or XPath).
- `html_table()` - parse an HTML table into a data frame.
- `html_text()` / `html_text2()` - extract a tag's text content.
- `html_name()` - extract a tag/element's name(s).
- `html_attrs()` - extract all attributes.
- `html_attr()` - extract attribute value(s) by name.

HTML, rvest, & xml2

```R
html =
'<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <p align="center">Hello world!</p>
    <br/>
    <div class="name" id="first">John</div>
    <div class="name" id="last">Doe</div>
    <div class="contact">
      <div class="home">555-555-1234</div>
      <div class="home">555-555-2345</div>
      <div class="work">555-555-9999</div>
      <div class="fax">555-555-8888</div>
    </div>
  </body>
</html>'
```

```R
read_html(html)
```

```
{html_document}
<html>
[1] <head>\n<meta http-equiv="Content-Type" cont
[2] <body>\n    <p align="center">Hello world!</
```

```R
read_html(html)[1]
```

```
$node
<pointer: 0x781d004d00>
```

Selecting elements

`html_element()` returns the first element matching a selector, and the `html_*()` helpers extract from it.

```r
read_html(html) |> html_element("p")
```

```
{html_node}
<p align="center">
```

```r
read_html(html) |> html_element("p") |> html_text()
```

```
[1] "Hello world!"
```

```r
read_html(html) |> html_element("p") |> html_name()
```

```
[1] "p"
```

```r
read_html(html) |> html_element("p") |> html_attrs()
```

```
   align
"center"
```

```r
read_html(html) |> html_element("p") |> html_attr("align")
```

```
[1] "center"
```

Selecting multiple tags

`html_elements()` returns every match as a node set. Unlike Beautiful Soup, the extraction functions are vectorized over node sets, so there is no need for a loop.

```r
read_html(html) |> html_elements("div")
```

```
{xml_nodeset (7)}
[1] <div class="name" id="first">John</div>
[2] <div class="name" id="last">Doe</div>
[3] <div class="contact">\n      <div class="home">555-555-1234</div>\n      ...
[4] <div class="home">555-555-1234</div>
[5] <div class="home">555-555-2345</div>
[6] <div class="work">555-555-9999</div>
[7] <div class="fax">555-555-8888</div>
```

```r
read_html(html) |> html_elements("div") |> html_text()
```

```
[1] "John"
[2] "Doe"
[3] "\n      555-555-1234\n      555-555-2345\n      555-555-9999\n      555-555-8888\n    "
[4] "555-555-1234"
[5] "555-555-2345"
[6] "555-555-9999"
[7] "555-555-8888"
```

```r
read_html(html) |> html_elements(".name") |> html_attr("id")
```

```
[1] "first" "last"
```

Same selectors

`rvest` uses the same CSS selectors as Beautiful Soup, so everything from earlier carries over.

```r
read_html(html) |> html_elements("p, #last")
```

```
{xml_nodeset (2)}
[1] <p align="center">Hello world!</p>
[2] <div class="name" id="last">Doe</div>
```

```r
read_html(html) |> html_elements("[class^=ho]")
```

```
{xml_nodeset (2)}
[1] <div class="home">555-555-1234</div>
[2] <div class="home">555-555-2345</div>
```

```r
read_html(html) |> html_elements("body > div")
```

```
{xml_nodeset (3)}
[1] <div class="name" id="first">John</div>
[2] <div class="name" id="last">Doe</div>
[3] <div class="contact">\n      <div class="home">555-
```

```r
read_html(html) |> html_elements(".contact div:not
```

```
{xml_nodeset (2)}
[1] <div class="work">555-555-9999</div>
[2] <div class="fax">555-555-8888</div>
```

`html_elements()` also supports XPath selectors as an alternative to CSS selectors, though we will prefer CSS selectors in

Missing elements

`html_element()` returns a missing node when nothing matches and the extraction functions then return `NA`, while `html_elements()` returns an empty node set and the extraction functions return an empty vector.

```r
page = read_html(html)
page |> html_element("span")
```

```
{xml_missing}
<NA>
```

```r
page |> html_element("span") |> html_text()
```

```
[1] NA
```

```r
page |> html_elements("span")
```

```
{xml_nodeset (0)}
```

```r
page |> html_elements("span") |> html_text()
```

```
character(0)
```

This is why `html_element()` is the right choice when extracting one field per item, as the output stays aligned with the inputs.

```r
page |> html_elements("body > div") |>
  html_element(".work") |> html_text()
```

```
[1] NA             NA             "555-555-9999"
```

`html_text()` vs `html_text2()`

```r
par = read_html(
  "<p>
    First sentence.
    Same line.<br>New line.
  </p>"
)
```

```r
par |> html_text()
```

```
[1] "\n    First sentence.\n    Same line.New line.\n  "
```

```r
par |> html_text() |> cat(sep="\n")
```

```
    First sentence.
    Same line.New line.
```

```r
par |> html_text2()
```

```
[1] "First sentence. Same line.\nNew line."
```

```r
par |> html_text2() |> cat(sep="\n")
```

```
First sentence. Same line.
New line.
```

Scraping with polite

The `polite` package handles `robots.txt`, delays, and caching for us while maintaining the three pillars of a polite session:

- seek permission
- take it slowly
- never ask twice

`bow()` reads the site's `robots.txt` and returns a session object, `nod()` moves to a new path within the site, and `scrape()` makes the request (this is equivalent to `read_html()` and returns a parsed HTML object).

```r
session = polite::bow("https://example.com")
polite::nod(session, "page/1") |> polite::scrape()
polite::nod(session, "page/2") |> polite::scrape()
```

`polite` uses the larger of the requested delay (5 seconds by default) and the crawl delay in `robots.txt`.

Graceful error handling

`purrr::possibly()` wraps a function so that errors return a default value instead of stopping, which is the `try` / `except` pattern from earlier packaged as a function.

```r
urls = c("https://www.google.com/search?q=duke",
         "https://www.google.com/maps")
```

```r
safe_read = purrr::possibly(read_html, otherwise = NULL)
urls |>
  purrr::map(safe_read)
```

`purrr::safely()` is similar but returns a list with `result` and `error` components, letting you inspect what went wrong.

```r
safe_read = purrr::safely(read_html)
results = urls |> purrr::map(safe_read)
```

```r
results |> purrr::map("result")  # successful results (NULL on failure)
results |> purrr::map("error")   # error objects (NULL on success)
```

Summary

Beautiful Soup and rvest

Task	Python	rvest
download a page	`requests.get(url).text`	`read_html(url)`
parse HTML	`BeautifulSoup(html, "html.parser")`	`read_html(html)`
select all matches	`select(css)`, `find_all()`	`html_elements(css)`
first match	`select_one(css)`, `find()`	`html_element(css)`, per input
no match	`[]`, `None`	empty node set / missing node
text	`get_text()`, `get_text(" ", strip=True)`	`html_text()`, `html_text2()`
tag name	`.name`	`html_name()`
all attributes	`.attrs`	`html_attrs()`
one attribute	`tag["href"]`, `tag.get("href")`	`html_attr("href")`
many elements	comprehension	vectorized
robots.txt	`urllib.robotparser`	`polite::bow()`
rate limiting	`time.sleep()`	`polite::scrape()`, `Sys.sleep()`
error handling	`try` / `except`	`possibly()`, `safely()`

Takeaways

- HTML is hierarchical, and scraping is the work of picking the elements we want out of that hierarchy and flattening them into a data frame.
- CSS selectors do the picking in both languages. Tools like SelectorGadget and the browser's inspector help find a selector that matches what we want and nothing else.
- Beautiful Soup's methods work on one `Tag` at a time and are combined with comprehensions, while `rvest`'s functions are vectorized over a set of elements.
- `find_all()` covers the cases a selector cannot, such as matching with a regular expression or an arbitrary function.
- Just because a page can be scraped does not mean it should be. Check `robots.txt` and the terms of service, slow down your requests, and expect some of them to fail.